

ES-6 (ECMA)

ECMAScript :

A Standard for Scripting Language like the one you are probably using –Javascript

3 Types of Variable

- Var , Let , Const Keywords in Js
- Var use : ES5
- Let & Const Use in :ES6
- Var Scope : Global & Local
- Let & Const : Blocked Scope

Ex:

```
Var x="Aspire";  
    x=" Multimedia";  
    Document.write(x);
```

```
Let x="Hello";  
    x=" Word";  
    Document.write(x);
```

```
Const x="Good Morning";  
    x=" Word"; ----- Wrong  
    Document.write(x);
```

-
- Var is a Global Scope for Window base (Let & const is not Global Scope)
 - Var

```
<script type="text/javascript">  
    var x="aspire";  
        console.log(x);  
        console.log(window);    or    console.log(window.x);  
</script>
```

- Let

```
<script type="text/javascript">  
    let x="aspire";  
        console.log(x);  
        console.log(window.x);  
</script>
```

- Const

```
<script type="text/javascript">
    const x="aspire";

    console.log(x);

    console.log(window.x);

</script>
```

Template Lierals

```
<script type="text/javascript">

    var x=`Aspire Multimedia " & Design`
    var y="Aspire Multimedia " & Design"  (not output)

    console.log(y);

</script>
```

Backstitch Program

```
<script type="text/javascript">

    let str="Aspire Multimedia";

    let str1="It Education Center";


    //console.log(str + " . " + str1 + " . ");

Or (concatenet function)

    console.log(`${str}. ${str1}.`);

</script>
```

Object Methods

```
<script>
    // Create Target Object
    const person1 = {
        firstName: "John",
        lastName: "Doe",
        age: 50,
        eyeColor: "blue"
```

```

    };

    // Create Source Object
    const person2 = {firstName: "Anne",lastName: "Smith"};

    // Assign Source to Target
    Object.assign(person1, person2);

    // Display Target
    let text = Object.entries(person1);
    console.log(person1,person2);
</script>

```

1. **Output :** {firstName: 'Anne', lastName: 'Smith', age: 50, eyeColor: 'blue'}

1. **age:** 50
2. **eyeColor:** "blue"
3. **firstName:** "Anne"
4. **lastName:** "Smith"
5. **[[Prototype]]:** Object

1. {firstName: 'Anne', lastName: 'Smith'}

1. **firstName:** "Anne"
2. **lastName:** "Smith"
3. **[[Prototype]]:** Object

Arrow Functions

```
const x = (x, y) => x * y;
```

```
console.log(5*5);
```

Old method

```
const addition = function (number) { return 10 + number; };
```

```
console.log(addition(20));
```

New method

```
const addition = number => 10 + number;
```

```
console.log(addition(20))
```

Binding this

```
let userDetails = {
  name: "Ramesh Khunt",
  Age : 35,
  Designation : "Engg.",
  printDetails: function() {

    console.log(this.Age)

  }
}
userDetails.printDetails()
```

Binding Call this

```
let userDetails = {
  name: "Ramesh Khunt",
  Age : 35,
  Designation : "Engg.",
  printDetails: function() {
    console.log(this.Age)
  }
}
userDetails.printDetails();

let userDetails2 = {
  name: "Anita Khunt",
  Age : 34,
  Designation : "BA",
  printDetails: function() {
    console.log(this)
  }
}
userDetails.printDetails.call(userDetails2);
```

Array Map

```
<script>
  const colors = ["red", "blue", "green"];
  const items = colors.map((color) => `<li> ${color} </li>`);
  console.log('items', items);
</script>
```

items : Array(3)

- 0: " red "
- 1: " blue "
- 2: " green "
- length: 3
- [[Prototype]]: Array(0)

Object Destructuring

```
<script>
// Create an Object
const person = {
    firstName: "John",
    lastName: "Doe",
    age: 50,
    eyeColor: "blue"
};

// Destructuring Assignment

let { firstName, age } = person;

console.log(name);
</script>
```

Spread Operator(...)

Ex – 1

```
const array1= [1, 2, 3];
const array2 = [4, 5, 6];
const combined = array1.concat(array2);
const newCombined = [...array1, ...array2];
console.log(combined, newCombined)

Output : [ 1, 2, 3, 4, 5, 6 ] [ 1, 2, 3, 4, 5, 6 ]
```

Ex—2

```
const address = {
street: "1195 Hart Street",
city: "Hartford",
country: "United States",
};

const fullAddress = {
...address,
state: 'Connecticut'
};
```

```
console.log(fullAddress)
```

```
1. {street: '1195 Hart Street', city: 'Hartford', country: 'United States', state: 'Connecticut'}
   1. city: "Hartford"
   2. country: "United States"
   3. state: "Connecticut"
   4. street: "1195 Hart Street"
   5. [[Prototype]]: Object
```

Classes

JavaScript Classes are templates for JavaScript Objects.

Use the keyword **class** to create a class.

Always add a method named **constructor()**:

Ex-1

```
<script>
    class Car {
        constructor(name, year) {
            this.name = name;
            this.year = year;
        }
    }

    const myCar1 = new Car("Ford", 2014);
    const myCar2 = new Car("Audi", 2019);
    console.log(myCar1.name + " " + myCar2.name);
</script>
```

Ex—2

```
<script>
class Car {
    constructor(name, year) {
        this.name = name;
        this.year = year;
    }
    age() {
        const date = new Date();
        return date.getFullYear() - this.year;
    }
}
```

```
}  
}  
const myCar = new Car("Ford", 2014);  
console.log("My car is " + myCar.age() + " years old.");
```

Class Inheritance

To create a class inheritance, use the **extends** keyword.

A class created with a class inheritance inherits all the methods from another class:

Ex—1

```
<script>  
class Car {  
    constructor(brand) {  
        this.carname = brand;  
    }  
    present() {  
        return 'I have a ' + this.carname;  
    }  
}  
class Model extends Car {  
    constructor(brand, mod) {  
        super(brand);  
        this.model = mod;  
    }  
    show() {  
        return this.present() + ', it is a ' + this.model;  
    }  
}
```

```
const myCar = new Model("Ford", "Mustang");

console.log(myCar);

</script>
```

Get & Set Method

```
<script>
  class Car {
    constructor(brand) {
      this.carname = brand;
    }
    get cnam() {
      return this.carname;
    }
    set cnam(x) {
      this.carname = x;
    }
  }

  const myCar = new Car("Ford");

  console.log(myCar);
</script>
```

Output : *Car {carname: 'Ford'}*

1. **carname:** "Ford"
2. **cnam:** (...)
3. **[[Prototype]]:** Object